

AN007: ASR 平台使用 qlllog 抓取日志

文件保密级别：（请勾选 ☒ ）

绝密 ☐

保密 ☐

公开 ☒

文件管控表

文件变更记录			
修订日期	最新版本	修订内容	编制
2025-02-20	V1.0	首次发行	William liu

1 目的

由于产品化之后通常没有预留 USB 口，这对售后抓取 LOG 带来了困难，同时越来越多的车厂要求集成抓 LOG 相关的服务，可以实现远程抓 LOG 分析。基于这些需求，移远开发了 qllogd 服务用来提供给客户或者前线 FAE，用于简化抓取 LOG 的目的。这个服务当前只有在 ASR1806 平台提供，需要更新 SDK 版本到 2024 年 12 月以后的版本。

2 基于 SDK 接口的方式

我们开发了一套 SDK 接口，方便客户集成到应用中，使用方法和其他移远提供的 SDK 接口一致。头文件位于 ql-sysroots/usr/include/ql-sdk/ql_log.h。例子代码参考 sample/test_sdk_api/m_log.c。

2.1 抓 LOG 的源:

```
#define QL_LOG_SOURCE_MODEM      0x01
#define QL_LOG_SOURCE_APP        0x02
#define QL_LOG_SOURCE_KERNEL     0x03
#define QL_LOG_SOURCE_DUMP       0x04
#define QL_LOG_SOURCE_SYSINFO    0x05
#define QL_LOG_SOURCE_GNSS       0x06
```

Modem 是指抓 CP LOG，和 CatStudio 抓取保存的日志相同，用于分析网络问题。

App 是指抓应用的 LOG，相当于 Logcat 的应用日志筛选了 system 和 main 这两个 buffer。

Kernel 是指抓内核 dmesg 日志。

Dump 是指收集系统 dump。但是抓 dump 需要特殊操作，下面详细介绍。

Sysinfo 是指定时收集系统信息。

Gnss 是指单收集 GNSS 的 NMEA 报文，注意这个命令不会主动打开 GNSS，需要使用 ql_gnss 相关 API 先打开 GNSS。

2.2 LOG 等级

```
#define QL_LOG_LEVEL_ERROR      1
#define QL_LOG_LEVEL_WARNING    2
#define QL_LOG_LEVEL_INFO       3
#define QL_LOG_LEVEL_DEBUG      4
```

ERROR 等级只输出错误级别的

WARNING 等级输出警告和错误级别的日志

INFO 等级输出 INFO/WARNING/ERROR 等级的日志

DEBUG 等级输出 DEBUG/INFO/WARNING/ERROR 等级的日志。

可见，ERROR 等级输出最少日志，DEBUG 等级输出最多日志。

2.3 初始化和去初始化

```
int ql_log_init(void);
```

```
int ql_log_deinit(void);
```

和其他 SDK 服务一样，使用之前需要初始化连接 ql_logd 服务，使用完毕之后需要断开服务。

2.4 启动和停止抓取 log 服务

```
int ql_log_start(int source, int flag);
```

```
int ql_log_stop(int source);
```

其中 source 是只 2.1 中描述的源, flag 为预留, 当前没有使用, 默认传入 0 即可。

注意:

1、ql_log_start 异步操作, 开启之后就该函数会立刻返回, 由 ql_logd 服务在后台开对应的线程来收集, 直到用户调用 ql_log_stop 关闭为止。

2、可以同时抓取多个不同的源, 只需要多次调用 ql_log_start 函数即可。

2.5 配置和获取配置

```
int ql_log_get_config(int source, ql_log_config_t *p_config);
```

```
int ql_log_set_config(int source, ql_log_config_t *p_config);
```

```
typedef struct {
    char    log_dir[QL_LOG_DIR_NAME_MAX];
    uint32_t log_level;
    uint32_t log_file_size;
    uint32_t log_file_num;
} ql_log_config_t;
```

其中:

log_dir 表示保存 log 的路径。

log_level 表示抓取 log 的等级。

log_file_size 表示单个 log 文件最大的大小, 单位是 MB。

log_file_num 表示最多能保存多少个日志文件。

注意:

1、对于 modem 日志来说, 由于 ASR 底层的限制, 目前只能支持 ERROR 和 DEBUG 两个等级。

2、对于 sysinfo 日志来说, 只有 log_file_num 和 log_dir 是有效的, 其他参数无意义。

3、这个配置只是保存到内存里面, 重启后会恢复默认值, 可以通过配置文件来修改默认的配置, 参考第 5 章的描述。

<特别注意>:

在 /system/etc/mrvl_tel_diag.cfg 文件里面有个配置项

```
#Default Media: 0-off, 1-usb (default), 2-sd, 3-tcp
```

```
default_media=3
```

这个配置项决定 modem log 是使用 tcp 模式, 还是使用 usb 模式, 所以发现 usb 连接不上 catstudio, 先检查这个配置, 把 default_media=1 然后保存, 重启生效。用于临时连接 USB 抓取 log 的场景。

3 基于 qllog 命令的方式

在上面 2 中提供了基于 SDK 的访问方式，但是需要用户进行开发应用程序。

```
qllog software v1.0 (2024-08-02) - a tool to capture and store logs for ASR platform

Usage:
qllog -s source [-p path] [-l level] [-z file-size] [-n file-num] [-t timeout]

Options:
-s, --log-source      set log source to capture (available source: modem, app, kernel, dump, sysinfo, gnss)
-p, --log-path        set log store path
-l, --log-level       set log level (1:error, 2:warning, 3:info, 4:debug)
-z, --log-file-size   set log file max size in MB
-n, --log-file-num    set log file max number
-c, --get-config      get log config
-m, --multi-source    set multi log source (bitmask: modem:0x02, app:0x04, kernel:0x08, dump:0x10, sysinfo:0x20, gnss:0x40)
-t, --timeout         set log capture timeout in second
-h, --help            print help message

Example:
qllog -s app -t 5
qllog -s kernel -t 5
qllog -s modem -p /mnt/mmcblk0p1/log/modem
qllog -m 0xE0 -t 5 (capture modem & app & kernel logs)
```

参数解析：

- s, --log-source 设置源 source，参考 2.1 的描述。
- p, --log-path 设置对应源的 LOG 保存路径。
- l, --log-level 设置 LOG 等级，参考 2.2 的表述。
- z, --log-file-size 设置单个 LOG 文件最大大小。
- n, --log-file-num 设置最多保存 LOG 文件数量。
- c, --get-config 获取对应源的配置信息。
- m, --multi-source 设置多个源同时收集 LOG。
- t, --timeout 设置采集 LOG 的时间，默认不退出。
- h, --help 帮助。

参考例子：

1、使用默认配置采集 10 秒 LOG 然后退出

```
qllog -s modem -t 10
```

```
qllog -s app -t 10
```

```
qllog -s kernel -t 10
```

.....

2、采集 10 秒应用 logcat 日志并且保存到 /oemdata 目录

```
qllog -s app -p /oemdata -t 10
```

3、后台采集应用日志并且循环保存到 5 个文件，每个文件最大 20MB，保存到 emmc 分区

```
qllog -s app -p /mnt/mmcblk0p1/log/app -z 20 -n 5 &
```

4、同时收集 modem，app，kernel 的 log 10 秒

```
qllog -m 0xE0 -t 10
```

其中

-m 模式下只允许 -t 参数，不允许其他配置。

-m 后面跟的是每个源的 bitmask（位图），多个源通过“与”的方式传递进来。

```
BITMASK_MODEM = 0x02
```

```
BITMASK_APP = 0x04
```

```
BITMASK_KERNEL = 0x08
```

```
BITMASK_DUMP = 0x10
```

BITMASK_SYSINFO = 0x20

BITMASK_GNSS = 0x40

例如上面需要的 modem/app/kernel 同时收集, 则为

BITMASK_MODEM | BITMASK_APP | BITMASK_KERNEL = 0x02 | 0x04 | 0x08 = 0x0E

4 抓取内核 Dump

抓 dump 比较特殊, 因为内核 dump 之后是会重启的, 需要特殊处理。其中 qllog 的做法就是在出现 dump 之后重启, qllog 判断当前为 dump 触发状态, 就把 dump 信息采集, 并且保存到默认配置路径, 做完之后就重启, 重新进入正常工作状态。

4.1 抓 AP dump

首先输入命令, 这个命令配置之后可以立刻生效, 这个时候只需要等待内核 dump 即可。

```
qllog -s dump -t 1
```

作为测试, 可以手动输入命令触发内核 dump:

```
echo p >/dev/ramdump_ctl
```

可以从日志中看到, 当 dump 发生之后系统重启进入 dump 模式, 然后 qllog 会自动收集 DUMP 到指定目录, 然后重启回正常工作模式。可以查看对应目录下 Dump log 已保存。

注意:

Dump 保存路径只能在配置文件里面设定, 参考第 5 章描述。

4.2 抓 CP dump

抓 CP dump 命令需要首先输入 AT 命令设置系统 Dump 之后能进入 dump 模式, 因为 modem 默认 dump 之后是静默重启的, 不会让系统 dump, 注意该命令是重启才生效。

```
at+aconfig="RDUP=1"
```

```
at+aconfig="EEHP=2"
```

然后重启到系统正常之后输入抓 log 命令

```
qllog -s dump -t 1
```

这个时候才可以触发 dump。例如输入命令模拟 CP dump:

```
echo a > /dev/acipc
```

触发之后系统重启进入 dump 模式, 这个时候 ql_logd 会收集 log 并且保存到指定路径。收集完毕之后会重启进入正常模式。

注意, 需要手动设置回正常模式, 因为 RDUP 和 EEHP 标志位是会保存的, 并不会自动恢复。

```
at+aconfig="RDUP=0"
```

```
at+aconfig="EEHP=1"
```

注意:

抓系统 Dump 建议用于临时分析问题使用, 不建议做进去系统版本。

5 配置文件

在 ql_logd 服务启动的时候, 会读取 /etc/ql_logd.conf 文件的配置作为默认配置。用户可以在 SDK 里面修改这个默认配置文件的参数, 通常建议产品化之后通过默认配置文件固化下来。默认的配置如下:

```
# Modem default log config
MODEM_LOG_PATH=/mnt/mmcblk0p1/log/modem
MODEM_LOG_LEVEL=4
MODEM_LOG_SIZE=100
MODEM_LOG_COUNT=4

# App default log config
APP_LOG_PATH=/mnt/mmcblk0p1/log/app
APP_LOG_LEVEL=4
APP_LOG_SIZE=100
APP_LOG_COUNT=4

# Kernel default log config
KERNEL_LOG_PATH=/mnt/mmcblk0p1/log/kernel
KERNEL_LOG_SIZE=10
KERNEL_LOG_COUNT=2

# Dump default log config
DUMP_LOG_PATH=/mnt/mmcblk0p1/log/dump
DUMP_LOG_COUNT=4
APP_DUMP_ENABLE=1
#log_temp_file_dir should not be the same as log_file_dir
APP_CORE_LOG_TEMP_FILE_DIR=/tmp
APP_CORE_FORMAT=coredump-%e-%p-%s-%t

# Sysinfo default log config
SYSINFO_LOG_PATH=/mnt/mmcblk0p1/log/sysinfo
SYSINFO_LOG_COUNT=10
SYSINFO_LOG_INTERVAL=600

# Gnss default log config
GNSS_LOG_PATH=/mnt/mmcblk0p1/log/gnss
GNSS_LOG_SIZE=100
GNSS_LOG_COUNT=4
```

其中:

- 1、PATH/LEVEL/SIZE/COUNT 的定义参考 2.5 章, 对应每个 LOG 源单独的配置。
- 2、SYSINFO_LOG_INTERVAL=600 是指每 600 秒收集一次 sysinfo, 同时 sysinfo 没有等级概念, 单个文件大小固定。

3、APP_DUMP_ENABLE 是指移远的 qlsdk 服务发生 dump 之后,可以收集对应的应用 dump log 用于分析问题,其中 APP_CORE_LOG_TEMP_FILE_DIR=/tmp 表示发生 dump 之后临时保存在 /tmp 目录,在 qllogd 检测到产生了 dump 文件之后会收集并且压缩打包放到 DUMP_LOG_PATH 指定的路径,并从 /tmp 目录删除,APP_CORE_FORMAT 是指定 dump 日志生成的格式。

4、modem 的 log level 只能设置 1 或者 4,对应 Error 和 Debug 等级。

5、内核 dmesg 日志不能设置等级。

6、所以的 log 输出格式以 logcat 格式输出。

6 建议使用场景

1、App 日志是用于分析应用问题,QLSDK 问题,网络问题,ECALL 问题,是必须的,建议在后台一直抓。

2、Kernel 日志用于分析内核问题,驱动问题,是必须的,建议在后台一直抓。

3、Modem 日志用于分析 modem 问题,一般是驻网,网络问题等,在 APP 日志已经分析看不出问题的情况就需要这个。但是注意这个 Modem 日志数据量非常大,所以建议不能一直抓,需要预留接口,在需要复现问题的时候再触发来抓。

4、sysinfo 日志这个是抓系统的基本信息,是间隔采集的,是反映系统的一些配置的,一般用于辅助看拨号问题。确认系统版本信息,常用 AT 命令查看 modem 状态等,文件不大,可以一直抓。

5、dump 这个是用来抓内核 dump 的,正常系统不能开,是需要抓 dump 复现问题的时候手动触发抓取。

6、gnss 日志这个是专门用来抓 NMEA 报文的,是需要分析 GNSS 定位,偏移等芯片等问题的时候分析使用,注意这里只包含 NMEA 的,而 GNSS 服务本身的日志是包含是 APP 类日志里面。

总结:

app 和 kernel 和 sysinfo 是必须提供的,可以覆盖大部分的问题分析场景。遇到比较困难的 modem 类问题,例如和基站交互,断网等,需要提供 modem 日志,遇到系统 dump 的问题分析才需要抓 dump。分析 gnss 定位相关问题,才需要提供 gnss nmea 的 log。

7 日志数量级评估

以下数据是指 qlsdk 原始版本,没有跑复杂网络服务的参考数据。用户实际跑复杂业务的场景下,日志的量级差异较大,建议客户实际测试为准。

APP: 100MB/7 小时

kernel: 1MB/24 小时

modem: 15MB/10 分钟 Error 精简等级,85M/10 分钟 debug 全量 Log 等级

sysinfo: 320KB/1 个文件,每 10 分钟 1 个。

GNSS NMEA: 100MB/24 小时